

CO_Homework 5_Verilog 1_2026/05/20

班級：電資二 (北聯/北醫)

學號：B812111001

姓名：張博翔

組別：Group 7、成員：113820028 張宏奕、114054812 陳半半(隨班附讀)

討論時間：2026/05/20 (三) 23:00

方式：線上討論

討論內容與心得：

- **全加器的多重建模與驗證**：本次實驗透過 RTL 資料流 (Dataflow) 與 Behavioral 行為級兩種方式實作 4-bit 全加器。我們不僅藉由由上而下的架構 (Top-Down) 加深了內部線路串聯與進位信號傳遞的硬體觀念，也體會到行為級語法 (如位元串接與算術運算) 在簡化代碼上的高效，並透過寫作 Testbench 成功驗證了兩者在功能與波形上的一致性。
- **(補充)架構優化與性能對比**：在漣波進位加法器 (RCA) 與超前進位加法器 (CLA) 的速度對比實驗中，我們透過模擬觀察到 CLA 在 Step 5 即迅速完成整體運算，而 RCA 仍受限於層層傳遞的進位延遲。這讓我們親身體會到優化電路架構對大幅提升運算速度的關鍵性。此外，計數除頻器 (Divide by 2 Counter) 的實作也讓我們進一步掌握了時序邏輯中非阻塞賦值 ($\leq$) 與 D 型反相器的底層行為。
- **非整數除頻器 (Divide by 3) 的挑戰與實作**：在非整數 (奇數) 除頻器的設計中，由於無法單純靠正沿觸發達到 50% 的工作週期 (Duty Cycle)，我們採用了更進階的技巧。透過設計兩個獨立的計數器分別在時脈正沿 (posedge) 與負沿 (negedge) 進行計數與狀態歸零，並將兩者的輸出進行邏輯組合 (如 OR 運算)。最終在 VSCode 中使用 Icarus Verilog 編譯並透過 GTKwave 觀察波形，成功驗證了輸出時脈頻率確實精準降為輸入時脈的 1/3，大幅深化了我們對時序邏輯電路 (Sequential Circuit) 與時脈域控制的理解。

會議照片：

Homework Q1 : 4-bit full adder

實作方式：

1. Icarus Verilog & GTK wave (vscode)
2. Xilinx (virtualbox)
3. Digital

Verilog HDL：

1. Icarus Verilog & GTK wave (vscode)：

Method 1：

```
1. // (RTL / Dataflow Modeling)
2. // =====
3. // sub_ckt : 1-bit full adder
4. // =====
5. module full_adder_1bit_rtl (
6.     input a,      // 輸入 A 1 bit
7.     input b,      // 輸入 B 1 bit
8.     input cin,    // 進位輸入
9.     output sum,   // 本位和
10.    output cout   // 進位輸出
11.);
12.    // Continuous Assignment
13.    // sum = a XOR b XOR cin
14.    //(a、b、cin 若為奇數個 1，則 XOR 為 1；偶數個 1，則 XOR 為 0，行為如加
    法，1+1 則會進位，本位為 0)
15.    assign sum = a ^ b ^ cin;
16.
17.    // cout = (a AND b) OR (cin AND (a XOR b))
18.    //(a、b、cin 互相 and 後 or 起來，代表只要有 2 個 bit 同時為 1 必發生進位，
    cout=1)
19.    assign cout = (a & b) | (cin & a) | (cin & b);
20. endmodule
21.
22. // =====
23. // Top module : 4-bit full adder (Top-Down Structural Hierarchy)
```

```

24.// =====
25.module adder_4bit_rtl (
26.    input [3:0] A,      // 4-bit 輸入 A
27.    input [3:0] B,      // 4-bit 輸入 B
28.    input cin,          // 進位輸入
29.    output [3:0] sum,    // 4-bit 和
30.    output cout         // 進位輸出
31.);
32.    // 宣告內部連線 (wire)，用於傳遞相鄰加法器之間的進位信號
33.    wire c1, c2, c3;
34.
35.    // 由上而下 Instantiation 4 個 1-bit 全加器
36.    // fa0: 處理最低位 (bit 0) 的加法
37.    full_adder_1bit_rtl fa0 (
38.        .a(A[0]), .b(B[0]), .cin(cin), .sum(sum[0]), .cout(c1)
39.    );
40.
41.    // fa1: 處理第 1 位的加法，進位輸入接自 c1
42.    full_adder_1bit_rtl fa1 (
43.        .a(A[1]), .b(B[1]), .cin(c1), .sum(sum[1]), .cout(c2)
44.    );
45.
46.    // fa2: 處理第 2 位的加法，進位輸入接自 c2
47.    full_adder_1bit_rtl fa2 (
48.        .a(A[2]), .b(B[2]), .cin(c2), .sum(sum[2]), .cout(c3)
49.    );
50.
51.    // fa3: 處理最高位 (bit 3) 的加法，最終進位輸出到頂層的 cout
52.    full_adder_1bit_rtl fa3 (
53.        .a(A[3]), .b(B[3]), .cin(c3), .sum(sum[3]), .cout(cout)
54.    );
55.
56.endmodule

```

Method 2 :

```

// (Behavioral Modeling)
// =====

```

```

// Top module : 4-bit full adder
// =====
module adder_4bit_behavioral (
    input [3:0] A,          // 4-bit 輸入 A
    input [3:0] B,          // 4-bit 輸入 B
    input cin,              // 進位輸入
    output reg [3:0] sum,    // 4-bit 和 (always 內賦值需為 reg)
    output reg cout         // 進位輸出
);

    // 觸發條件：當 A、B 或 cin 的數值發生任何改變時，執行區塊內部電路
    always @(A or B or cin) begin
        // procedure assignment，利用位元串接與算術加法
        // 將 1-bit 的 cout 與 4-bit 的 sum 拼接成 5-bit 的左值來接收加法結果
        {cout, sum} = A + B + cin;
    end
endmodule

```

testbench：

```

`timescale 1ns/100ps    // 設定時間單位為 1ns，時間精確度為 100ps
`include "fulladder4.v" // 導入 4-bit full adder 模組
`include "fulladder4_2.v" // 導入 4-bit full adder 模組
module fa4_tb;

    // 宣告輸入訊號（接收輸入的訊號為 register 型態，如此一來才能將訊號 hold
    住而非一斷開就高阻抗狀態，對電路不好）
    reg [3:0] A;
    reg [3:0] B;
    reg cin;

    // 宣告輸出訊號（接收輸出的訊號為 wire 型態）
    wire [3:0] sum_rtl;
    wire cout_rtl;
    wire [3:0] sum_behav;
    wire cout_behav;

```

```

// 實例化 RTL full adder (被測單元 UUT1)
adder_4bit_rtl uut_rtl (
    .A(A), .B(B), .cin(cin), .sum(sum_rtl), .cout(cout_rtl)
);

// 實例化 Behavioral full adder (被測單元 UUT2)
adder_4bit_behavioral uut_behav (
    .A(A), .B(B), .cin(cin), .sum(sum_behav), .cout(cout_behav)
);

// 5. 測試信號
initial begin
    // --- Icarus Verilog 波形導出 ---
    $dumpfile("full_adder_4bit_wave.vcd"); // 指定輸出的波形檔名
    $dumpvars(0, fa4_tb);                // 輸出此模組下的所有信號變化

    // --- 終端機即時視窗監看設定 ---
    $monitor("Time=%0dns | A=%b B=%b cin=%b | RTL_Sum=%b RTL_cout=%b
| Behav_Sum=%b Behav_cout=%b",
        $time, A, B, cin, sum_rtl, cout_rtl, sum_behav,
cout_behav);

    // --- 測試測資開始 ---
    // 測資 1: 0 + 0 + 0 = 0
    A = 4'b0000; B = 4'b0000; cin = 1'b0;
    #5; // 延遲 5 個單位

    // 測資 2: 3 + 5 + 0 = 8 (無進位)
    A = 4'b0011; B = 4'b0101; cin = 1'b0;
    #5;

    // 測資 3: 5 + 10 + 1 = 16 (產生最高位進位, sum 為 0)
    A = 4'b0101; B = 4'b1010; cin = 1'b1;
    #5;

    // 測資 4: 15 + 15 + 1 = 31 (最大值全滿)
    A = 4'b1111; B = 4'b1111; cin = 1'b1;

```

```

#5;

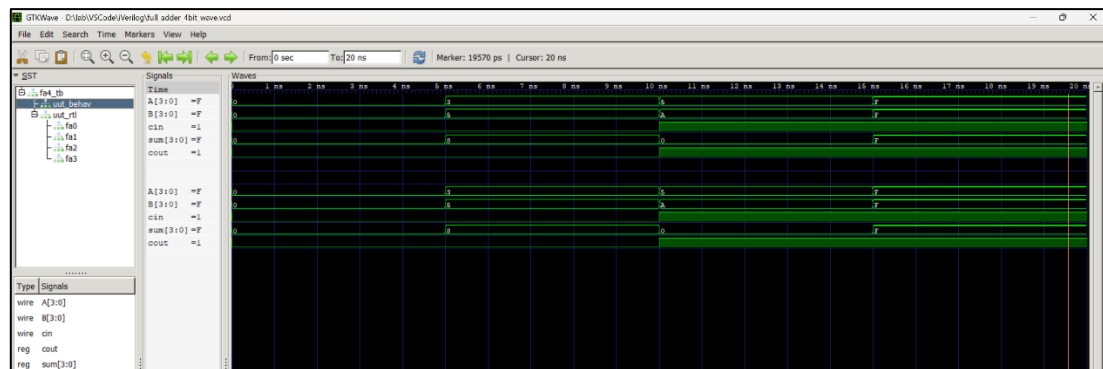
// 結束模擬
$finish;

end

endmodule

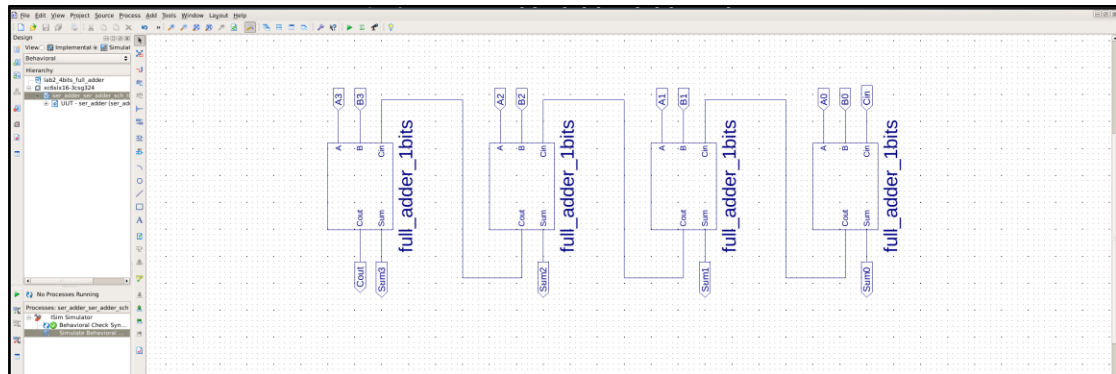
```

Simulation Result :

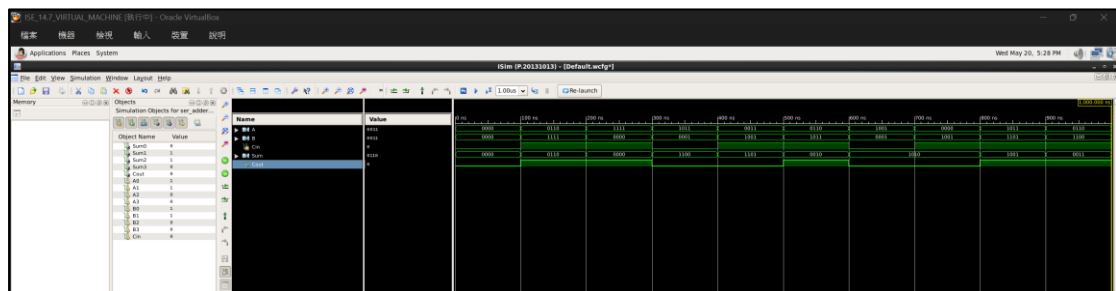


2. Xilinx (virtualbox) :

Schematic :



Simulation Result :



3. 加法器結果怎麼看？（有號數判斷）

- 這次實驗做的是 4-bit 的加法減法。4-bit 能表達的數字範圍是-8 到+7
 - ◆ 相加變小（正確）：輸入-5 加 -2，電路輸出 1101。因為 1101 在電腦二進位裡代表 -7。
 - ◆ 減去負數變大（正確）：輸入 0 減去-7，電路輸出 0111（也就是 +7）。
 - ◆ 溢位 Overflow: 輸入 -8\$ 加 -2\$ 時，數學上應該是 -10\$，但因為超出了電路能表達的範圍（-8~+7），結果二進位反而變成 +6。

4. Digital (補充)

4-bit 加減法器，由 Cin 控制加法或減法，將 Cin 與其中一串輸入做 XOR，當 Cin=1，該數值 0 會因為與 Cin=1 XOR 變 1；反之數值 1 會因為與 Cin=1 XOR 變 0，等同取一補數，再因為加入 Cin，等同取二補，執行減法運算。

5. 依據全加器的邏輯概念建立真值表，再轉換為卡諾圖。

表. Full adder 真值表

X	Y	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$X \setminus Y C_{in}$	00	01	11	10
0	0	1	0	1
1	1	0	1	0

圖. Sum bit 卡諾圖

$X \setminus Y C_{in}$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

圖. Cout bit 卡諾圖

6. 參考卡諾圖將布林函式化簡，使用 Digital 完成連接進行電路模擬，測試不同輸入組合，確認輸出結果符合原先表示式後將其作為模組。

$$\begin{aligned}
 Sum &= X'Y'C_{in} + X'YC'_{in} + XY'C'_{in} + XYC_{in} \\
 &= X'(Y'C_{in} + YC'_{in}) + X(Y'C'_{in} + YC_{in}) \\
 &= X'(Y \oplus C_{in}) + X(Y \oplus C_{in})' = X \oplus Y \oplus C_{in}
 \end{aligned}$$

圖. Sum bit 表示式

$$\begin{aligned}
 C_{out} &= X'YC_{in} + XY'C_{in} + XYC'_{in} + XYC_{in} \\
 &= (X'YC_{in} + XYC_{in}) + (XY'C_{in} + XYC_{in}) + (XYC'_{in} + XYC_{in}) \\
 &= YC_{in} + XC_{in} + XY
 \end{aligned}$$

圖. Cout bit 表示式

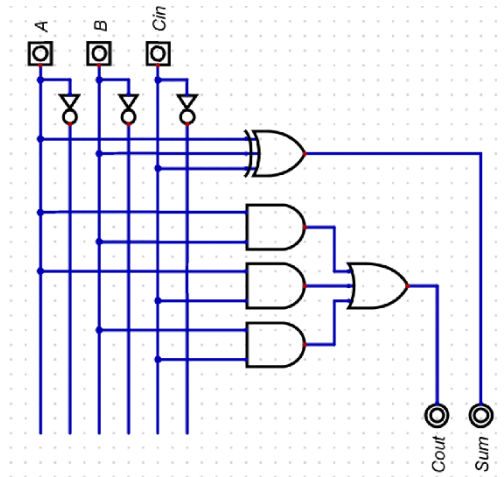


圖. 1-bit Full adder 邏輯電路

- 將模組串聯合成 4-bit Ripple Carry Adder，為了實現加減法器(Adder-Subtractor)，需讓 Cin 與其中一組數值進行 XOR 運算，當 Cin=0，兩數值直接相加；當 Cin=1，則與 Cin XOR 運算的數值會取 1 補數，加上 Cin 取 2 補數，達成減法效果，確認輸出結果符合預期後將其作為模組。

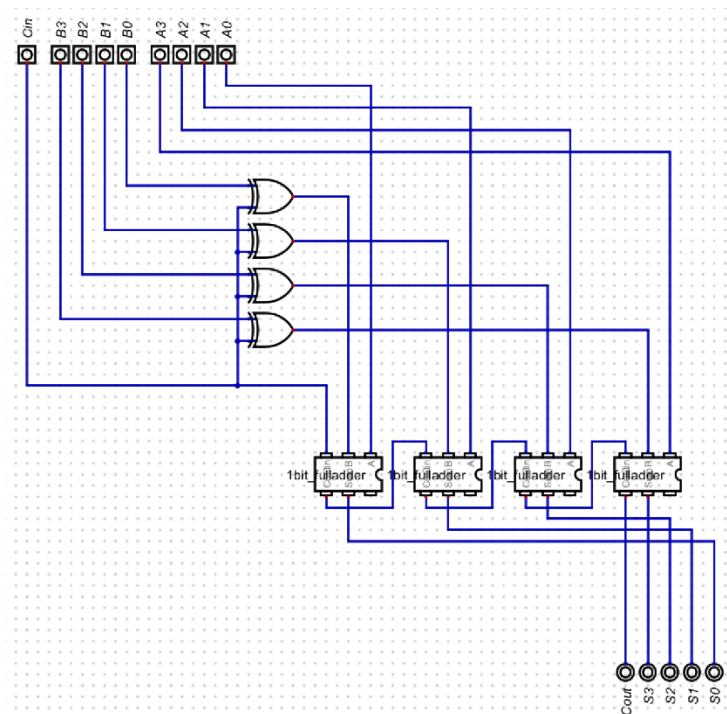


圖. 4-bit Adder-Subtractor 邏輯電路

8. 利用 Digital 的真值表生成 SOP 電路功能產出七段顯示器解碼電路，由於輸出為 4-bit 有號數，因此真值表範圍應包含 0~7、-1~-8，確認後作為模組。

表. 七段顯示器真值表

$S3$	$S2$	$S1$	$S0$	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	0	0
1	0	1	0	1	0	1	1	1	1	1
1	0	1	1	1	0	1	1	0	1	1
1	1	0	0	0	1	1	0	0	1	1
1	1	0	1	1	1	1	1	0	0	1
1	1	1	0	1	1	0	1	1	0	1
1	1	1	1	0	1	1	0	0	0	0

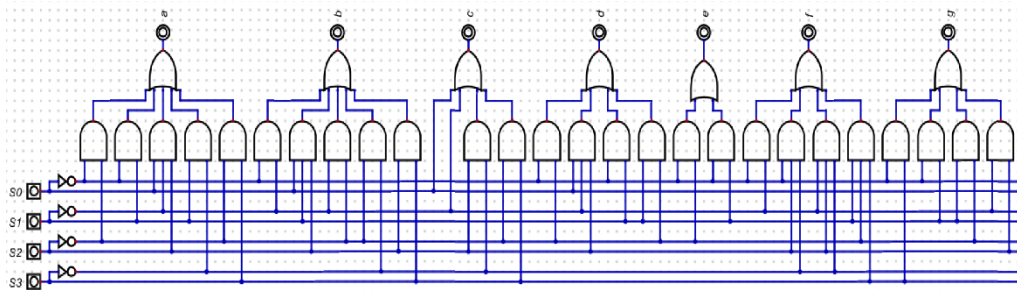


圖. 七段顯示器邏輯電路

9. 開啟 Embedded Circuit 將各模組串接，並且運用兩組數值 MSB 與 Sum MSB 判斷溢位，若兩組數值 MSB 相同但相加後 Sum MSB 變號則發生溢位。運用 Digital 內建的七段顯示器搭配進行模擬(由於 FPGA 的七段顯示器為共陽型，需要將 Digital 設定為 Common Connection: Anode)，測試不同輸入組合，確認輸出結果符合原先表示式後產出 Verilog HDL 檔案。

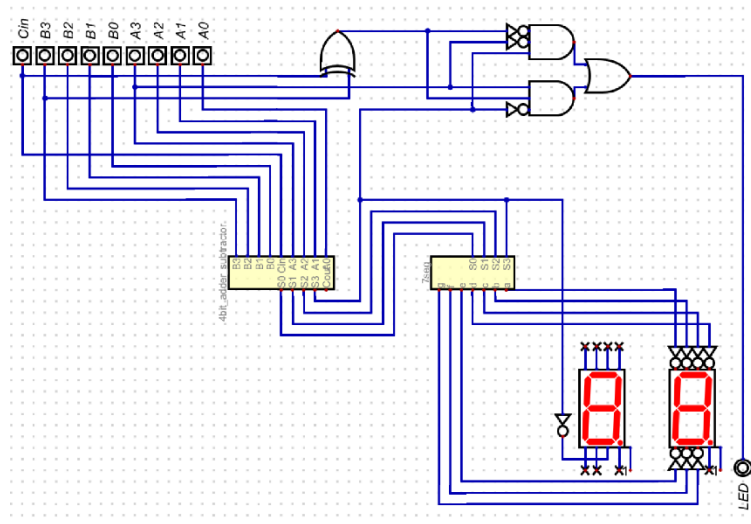


圖. 七段解碼器模擬電路圖

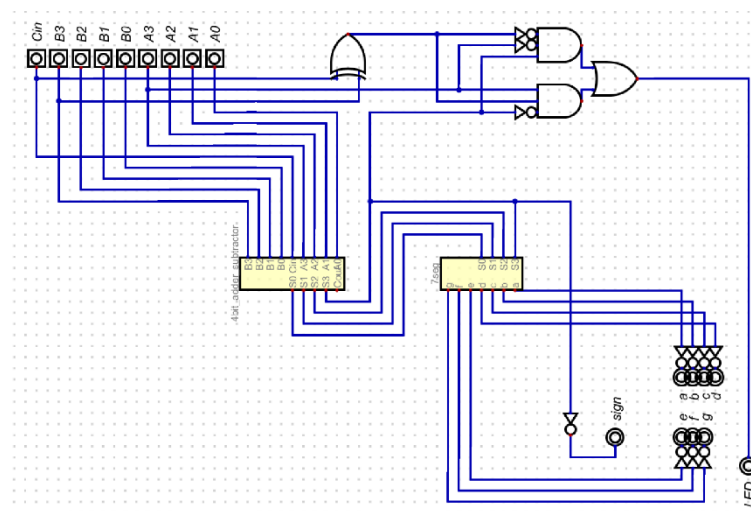


圖. 七段解碼器輸出電路圖(Verilog HDL 產出)

開啟 Quartus 建立專案，導入 Verilog HDL，修改內容檔名後進行第一次編譯，待變數讀入後定義腳位並再次編譯，完成後進行 FPGA 燒錄，並進行結果測試。

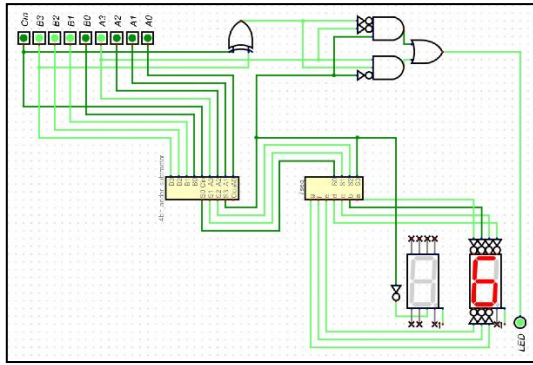


圖. Input:1000(A)+1110(B) / Output:(1)0110

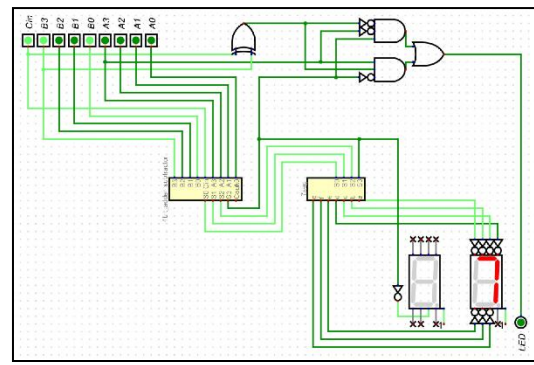


圖. Input:0000(A)-1001(B) / Output:0111

Overflow

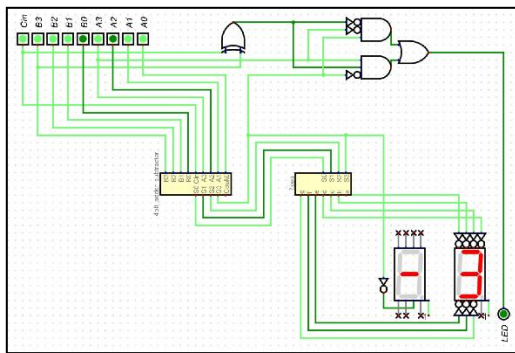


圖. Input:1011(A)-1110(B) / Output: 1101

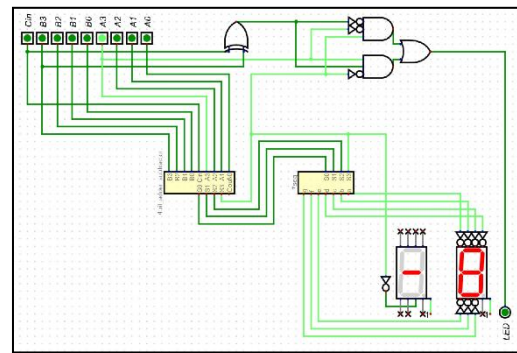


圖. Input:1000(A)+0000(B) / Output:1000

七段解碼電路：Input A($A_3A_2A_1A_0$)、B($B_3B_2B_1B_0$)、Control C(C_{in})，Output a、b、c、d、e、f、g
(以下為方便判斷輸出以計算結果代替)。(以下結果均符合模擬預測)

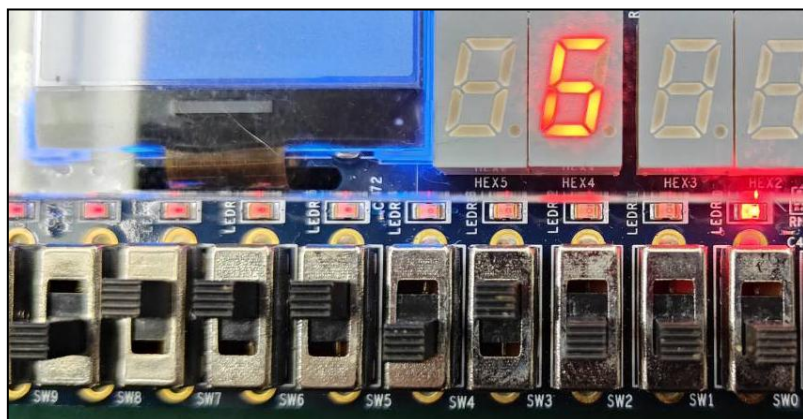


圖. Input:1000(A)+1110(B) / Output:(1)0110 overflow

兩者速度差異在 Step 5 最為明顯，此時 CLA 已完成整體運算，而 RCA 仍處於第三級全加器的進位傳遞階段。實驗結果證明，CLA 領先 RCA 四個 Step，成功驗證了 CLA 在多位元運算中，能透過平行產生進位的邏輯克服 RCA 的進位傳遞延遲，提升運算速度。

Carry-Lookahead adder 與 Ripple Carry adder 速度比較：Calculate : 0011+0101 Result : 1000

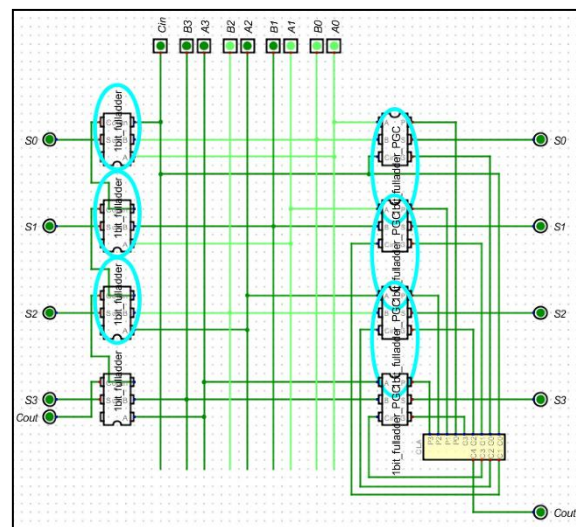


圖. RCA(左) : (0)0000 / CLA(右) : (0)0000 Step1

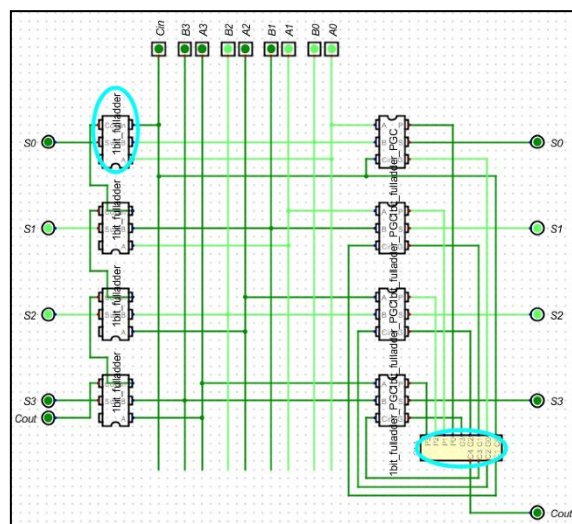


圖. RCA : (0)0110 / CLA : (0)0110 Step2

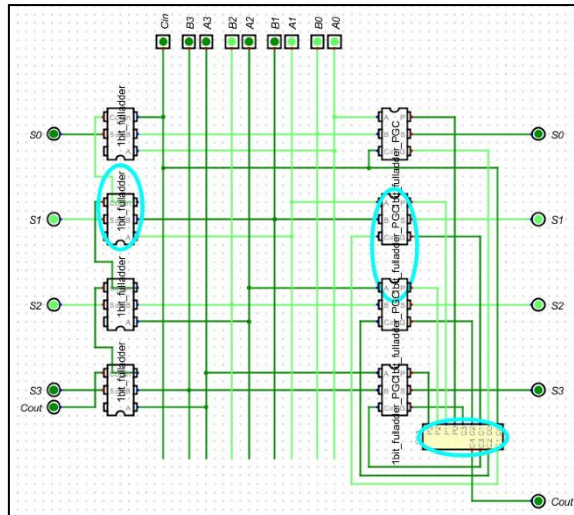


圖. RCA : (0)0110 / CLA : (0)0110 Step3

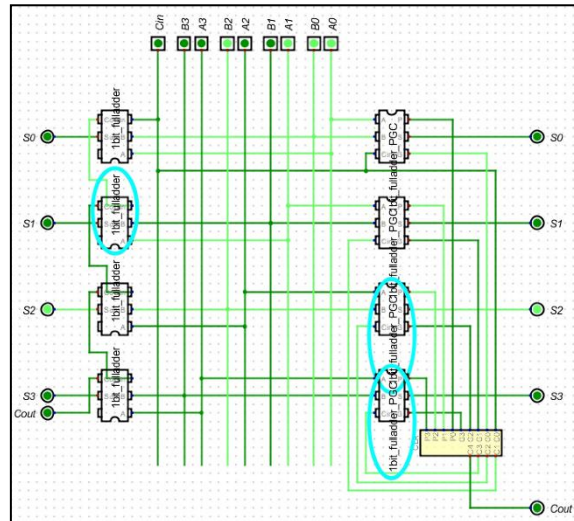


圖. RCA : (0)0100 / CLA : (0)0100 Step4

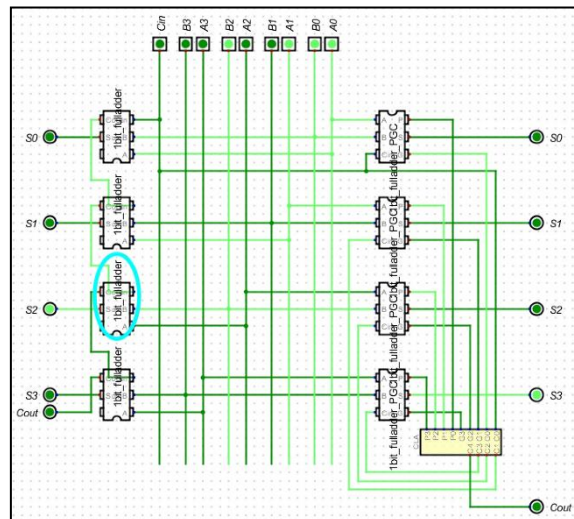


圖. RCA : (0)0100 / CLA : (0)1000 Step5

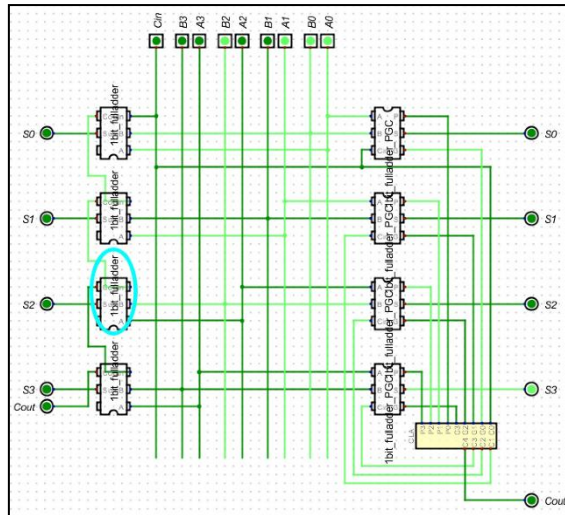


圖. RCA : (0)0000 / CLA : (0)1000 Step6

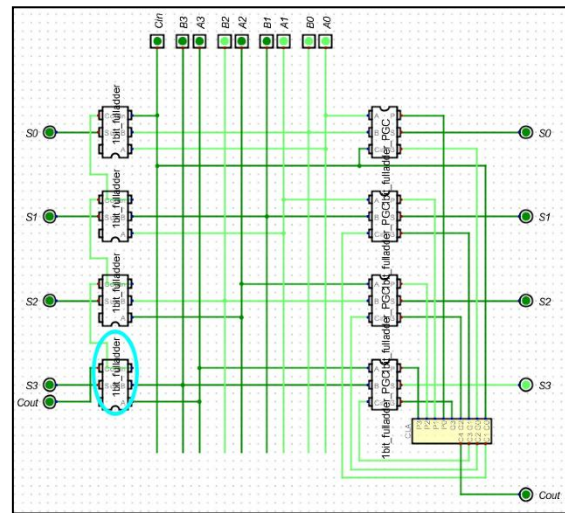


圖. RCA : (0)0000 / CLA : (0)1000 Step7

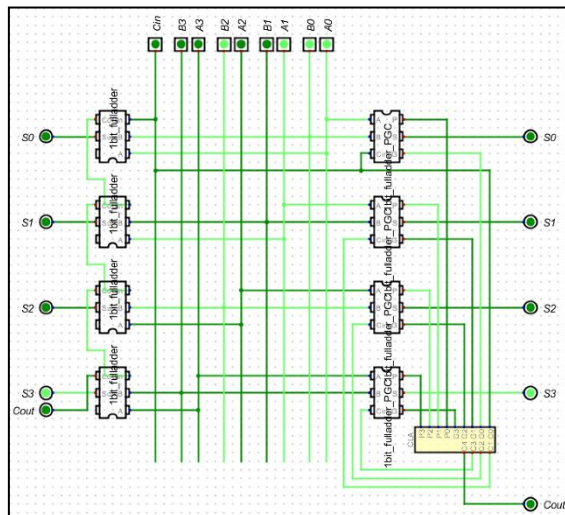


圖. RCA : (0)1000 / CLA : (0)1000 Step8

波形解釋

1. 測資切換週期:

本次 testbench 中，每一組輸入測資皆間隔 #5ns 進行切換。因此在波形中可觀察到 A、B、cin 每隔 5ns 改變一次，並對應產生新的加法輸出結果。

2. 輸出判讀方式:

4-bit 全加器的輸出需同時觀察 {cout, sum}。其中 sum 為低 4 位元的加法結果，cout 則為第 5 位進位輸出。若只觀察 sum，可能會誤判超過 4-bit 表示範圍的結果。

3. 第一組測資:

當 $A = 0000$ 、 $B = 0000$ 、 $cin = 0$ 時，加法結果為 0。因此波形中可觀察到 $sum = 0000$ 、 $cout = 0$ ，完整輸出為 {0,0000}。

4. 第二組測資:

當 $A = 0011$ 、 $B = 0101$ 、 $cin = 0$ 時，代表 $3 + 5 + 0 = 8$ 。因此波形中可觀察到 $sum = 1000$ 、 $cout = 0$ ，完整輸出為 {0,1000}。

5. 第三組測資:

當 $A = 0101$ 、 $B = 1010$ 、 $cin = 1$ 時，代表 $5 + 10 + 1 = 16$ 。由於 16 已超過 4-bit 的 sum 可直接表示範圍，因此低 4 位元為 0000，並由 cout 輸出進位 1。波形中可觀察到 $sum = 0000$ 、 $cout = 1$ ，完整輸出為 {1,0000}。

6. 第四組測資:

當 $A = 1111$ 、 $B = 1111$ 、 $cin = 1$ 時，代表 $15 + 15 + 1 = 31$ 。31 以 5-bit 表示為 11111，因此波形中可觀察到 $sum = 1111$ 、 $cout = 1$ ，完整輸出為 {1,1111}。

7. RTL 與 Behavioral 結果比較:

波形中同時顯示 RTL/Dataflow 與 Behavioral 兩種建模方式的輸出結果。當每組輸入改變後，兩者皆能得到相同的 sum 與 cout，代表兩種 Verilog 建模方式在功能上完全一致，成功驗證 4-bit 全加器的正確性。

Homework Q2 : Divide by 2 Counter

實作方式：

1. Icarus Verilog & GTK wave (vscode)

Verilog HDL：

10. Icarus Verilog & GTK wave (vscode)：

```
2. // Divide by 2 Counter
3. // 定義模組
4. module clk_div (clk_in, enable, reset, clk_out);
5.     // Port Declaration
6.     input clk_in ;    // 原始高頻時脈
7.     input reset ;    // 重置訊號 (Active High)
8.     input enable ;   // 致能訊號 (電路啟用)
9.     output clk_out ; // 除頻後時脈 (頻率為 clk_in 的一半)
10.
11.    // data type
12.    wire clk_in ;    // 預設為 wire，代表接收外部連續訊號
13.    wire enable ;
14.
15.    // Internal Registers
16.    reg clk_out ;    // always block 中賦值必須宣告為 reg 類型
17.
18.    // positive edge 時觸發
19.    always @ (posedge clk_in) begin
20.        if (reset)
21.            // 當同步重置訊號為 1 時，將輸出時脈強制清零
22.            // 採 non blocking (<=) 以符合 sequential circuit (D 型反相
            // 器) 的硬體行為
23.            clk_out <= 1'b0;
```

```

24.
25.     else if (enable)
26.         // 當不重置且啟用訊號為 1 時，在每個時脈正沿將輸出暫存器進行
           「邏輯反相 (!) / toggle」
27.         // 由於兩個正沿才完成一次「高 > 低 > 高」的完整週期，因此達到除
           以 2 的效果
28.         clk_out <= ! clk_out ;
29.     end
30.endmodule

```

testbench :

```

`timescale 1ns/100ps    // 設定模擬的時間單位為 1ns，時間精度/解析度為 100ps
`include "Lab01.v"       // 導入 Divide by 2 Counter 模組

module clk_div_tb;
    reg clk_in;
    reg reset;
    reg enable;

    wire clk_out;

    // Instantiate the Module
    // 將定義好的 clk_div 電路實例化為 INST1
    clk_div INST1 (
        .clk_in(clk_in),    // 將 testbench 的 clk_in 接入電路的 clk_in
port
        .enable(enable),    // 將 testbench 的 enable 接入電路的 enable
port
        .reset(reset),      // 將 testbench 的 reset 接入電路的 reset port
        .clk_out(clk_out)   // 將電路輸出的 clk_out 接回 testbench 的
clk_out
    );

    // Clock Generator
    initial clk_in = 1'b0;    // 將輸入時脈初始化為低電位 0

```

```

    //initial clk_out = 1'b0;      // 將輸出時脈初始化為低電位 0，避免出現
    高阻抗狀態，不穩

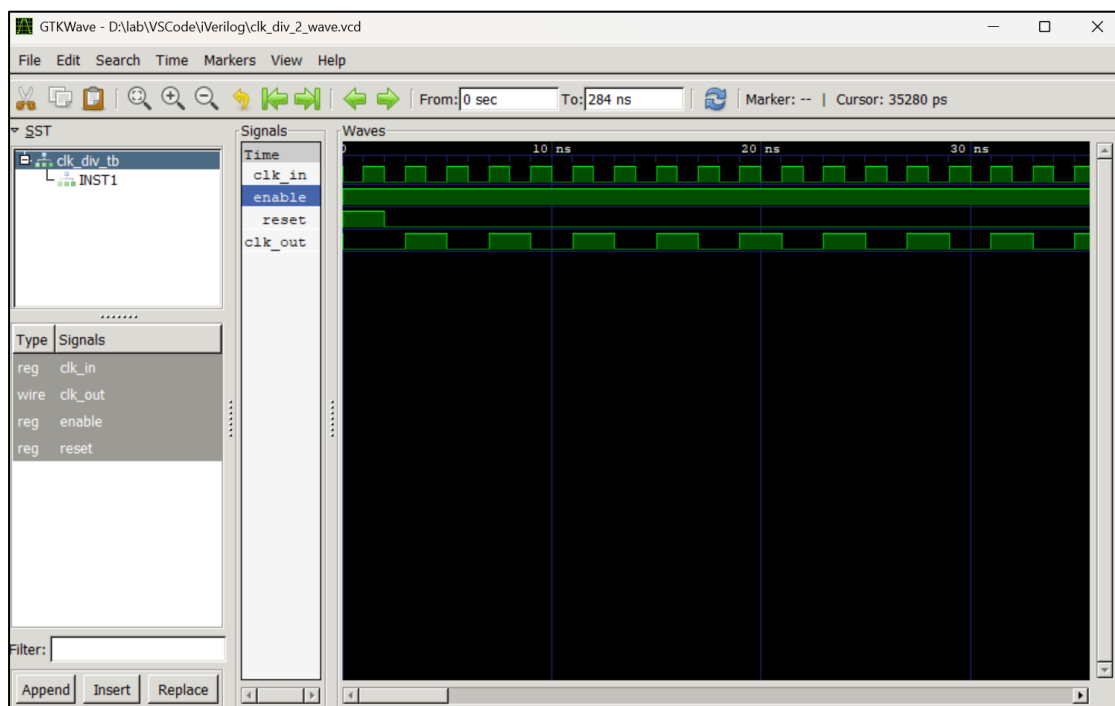
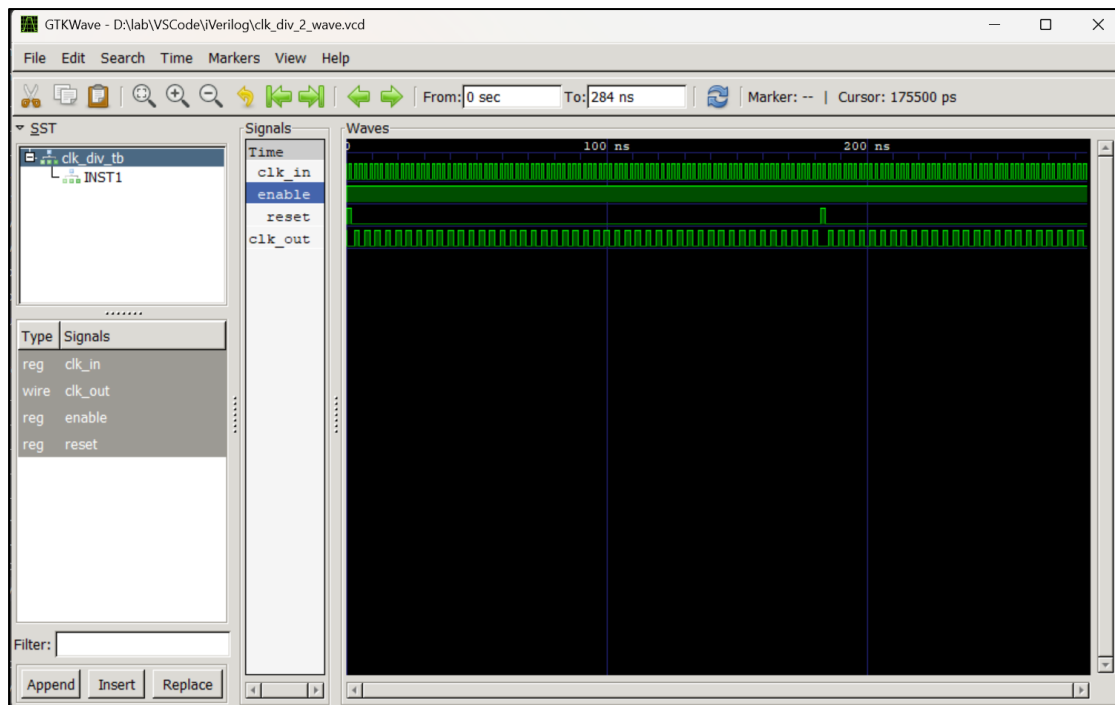
    always #1 clk_in = ~clk_in; // 每隔 1 個時間單位 (1ns)，將 clk_in 反
    相一次，產生週期為 2ns 的方波

    // Test Stimulus
    initial begin
        // --- Icarus Verilog 波形導出 ---
        $dumpfile("clk_div_2_wave.vcd"); // 指定輸出的波形檔名
        $dumpvars(0, clk_div_tb);       // 輸出此模組下的所有信號變化

        INST1.clk_out = 1'b0; // *強制在第 0 奈秒將 INST1 內部的 clk_out
        暫存器初始化為 0，消除開機到第一個 posedge clk 抵達前的高阻抗狀態
        reset = 1'b1;        // 模擬開始：先將重置訊號拉高，使電路進入確定
        的初始狀態
        enable = 1'b1;       // 將啟用訊號拉高，致能除頻電路
        #2 reset = 1'b0;     // 延遲 2 個時間單位 (2ns) 後，將重置訊號放
        開 (拉低)，電路開始正常除頻運作
        #180 reset = 1'b1;   // 正常運作 180ns 後，再次拉高重置訊號，用以測
        試中途重置電路的硬體行為
        #2 reset = 1'b0;     // 延遲 2ns 後再次釋放重置，讓電路重新恢復除
        頻
        #100 $finish;       // 當模擬時間達到 124ns 時，自動呼叫系統任務
        結束模擬，防止無限模擬崩潰
    end
endmodule

```

Simulation Result :



波形解釋：

1. 重置期 (0ns - 2ns)：當 `reset = 1` 時，不論 `clk_in` 極性如何，`clk_out` 被強制鎖定在低電位 0。
2. 正常除頻期 (2ns 之後)：
 - 觀察 `clk_in` 的第一個上升沿 (如 3ns 處)，此時 `clk_out` 從 0 翻轉 (Toggle) 為 1。

- 觀察 `clk_in` 的第二個上升沿（如 5ns 處），`clk_out` 從 1 翻轉回 0。
- 由此可清楚看到，`clk_in` 跑了 2 個完整週期，`clk_out` 才跑完 1 個完整週期（寬度恰為 `clk_in` 的 2 倍），成功驗證了除以 2 的硬體行為。

11. 除以 2 除頻器 (Divide by 2) 設計概念與用途:

- 設計概念: 利用時序邏輯中的 D 型反相器架構。電路僅在輸入時脈的正沿 (posedge clk_in) 進行觸發。當致能訊號 (enable) 為高電位時, 輸出暫存器在每次觸發時便執行一次「邏輯反相 (Toggle, !clk_out)」。由於輸出需要經歷兩個輸入時脈的正沿才能完成一次完整的「高 > 低 > 高」週期, 因此輸出頻率精準地被調製為輸入的一半。
- 預期結果: 輸入心跳跳了兩下, 輸出跳一下, 心跳速度被砍半 (頻率除以 2)。
- 應用:
 - ◆ 微處理器內部需要將主要系統時脈 (System Clock) 砍半給特定外設 (如低速加載計時器、傳輸速率產生器) 的場景。
 - ◆ 計數器 (Counter) 的二進位最低位元 (LSB) 訊號生成。

Homework Q3 : Divide by 3 Counter

實作方式:

1. Icarus Verilog & GTK wave (vscode)

Verilog HDL :

12. Icarus Verilog & GTK wave (vscode) :

```
1. // Divide by 3 Counter
2. // 定義
3. module ClockBy3(clk, reset, clk_out);
4.     // Port Declaration
5.     input clk;           // 原始時脈
6.     input reset;         // 同步重置訊號 (高電位有效)
7.     output clk_out;      // 除以 3 且具備 50% duty 的輸出時脈
8.
9.     // 宣告兩個 2 位元的暫存器 (範圍 0 到 3), 分別用於正沿與負沿的計數
10.    reg [1:0] pos_count, neg_count;
11.
12.    // Continuous Assignment
13.    // 當正沿計數器「或」負沿計數器的數值為 2 時, 輸出時脈即為高電位。
```

```

14.    // 透過這兩個錯開半個週期的計數器進行波形疊加(OR)，即可完美調製出 50%
    Duty Cycle 的波形！
15.    assign clk_out = ((pos_count == 2) | (neg_count == 2));
16.
17.    // 正沿計數器：獨立的循序邏輯電路，僅在時脈的上升沿觸發
18.    always @(posedge clk) begin
19.        if (reset)
20.            pos_count <= 0;                // 重置時，將正沿計數器清零
21.        else if (pos_count == 2)
22.            pos_count <= 0;                // 計數達到 2 時（已歷經 0, 1,
    2 共三個狀態），將計數器歸零重新計數
23.        else
24.            pos_count <= pos_count + 1; // 其餘情況，計數器遞增 1
25.    end
26.
27.    // 負沿計數器：僅在時脈的下降沿觸發
28.    always @(negedge clk) begin
29.        if (reset)
30.            neg_count <= 0;                // 重置時，將負沿計數器清零
31.        else if (neg_count == 2)
32.            neg_count <= 0;                // 計數達到 2 時，將計數器歸零重
    新計數
33.        else
34.            neg_count <= neg_count + 1; // 其餘情況，計數器遞增 1
35.    end
36. endmodule

```

testbench：

```

`timescale 1ns/100ps // 設定時間單位為 1ns，時間精度為 100ps
`include "Lab02.v"     // 導入 Divide by 3 Counter 模組

module clkdiv3_tb;
    reg clk, reset;
    wire clk_out;

    // Instantiate the Module
    // 將定義好的 ClockBy3 電路實例化為 INST

```

```

ClockBy3 INST (
    .clk(clk),
    .reset(reset),
    .clk_out(clk_out)
);

// 時脈產生電路
initial clk = 1'b0;          // 初始化時脈訊號為 0
always #5 clk = ~clk;        // 每 5ns 反相一次，產生一個週期為 10ns（頻率
100MHz）的標準輸入時脈

// 測試
initial begin
    // 初始化 reset
    reset = 1'b0;
    // 初始化 clk_out 的暫存器
    INST.pos_count = 2'b00;
    INST.neg_count = 2'b00;

    #5 reset = 1'b1;          // 模擬開始 5ns 後，拉高重置訊號，初始化內部電
路狀態
    #10 reset = 1'b0;         // 再過 10ns 後，放開重置訊號，讓電路開始正常
除 3 運作
    #500 $finish;             // 當模擬時間達到 500ns 時，自動呼叫系統任務結
束模擬，防止無限模擬崩潰
end

// 系統監視文字輸出 (System Monitor)
initial begin
    // 每當 clk、reset 或 clk_out 的狀態發生跳變時，自動在 ModelSim 終端
機列印目前的數值，方便文字 debug
    $monitor("Time=%0d ns, clk=%b, reset=%b, clk_out=%b", $time,
clk, reset, clk_out);
end

// VCD Waveform Dump
initial begin
    $dumpfile("clk_div_3_wave.vcd"); // clk_div_3_wave.vcd

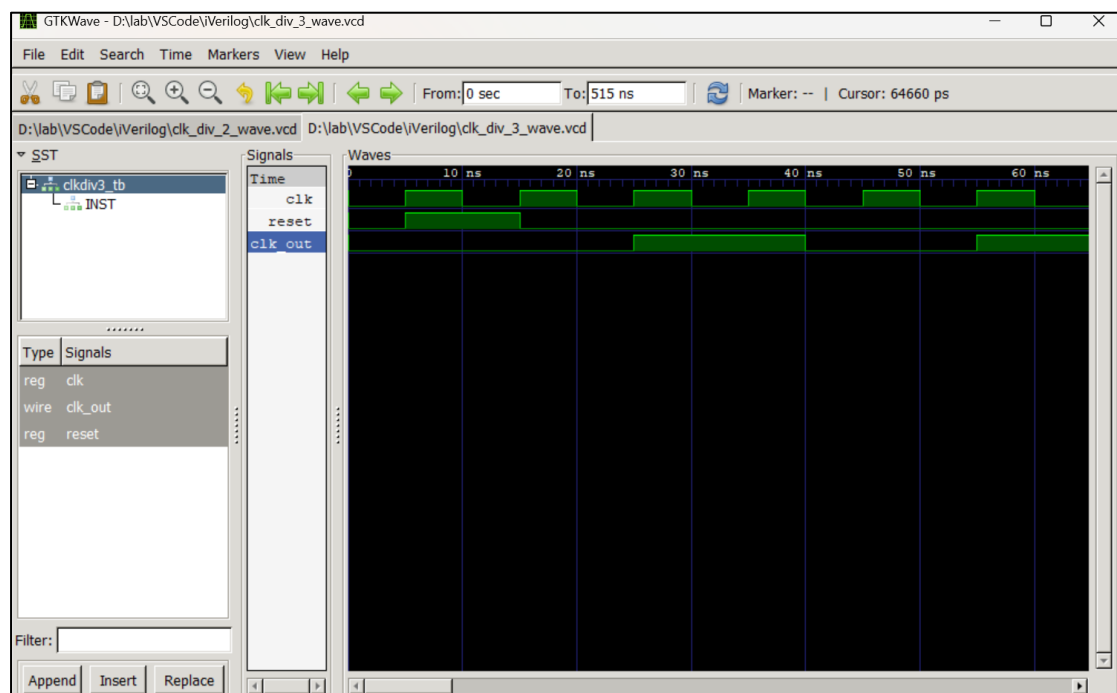
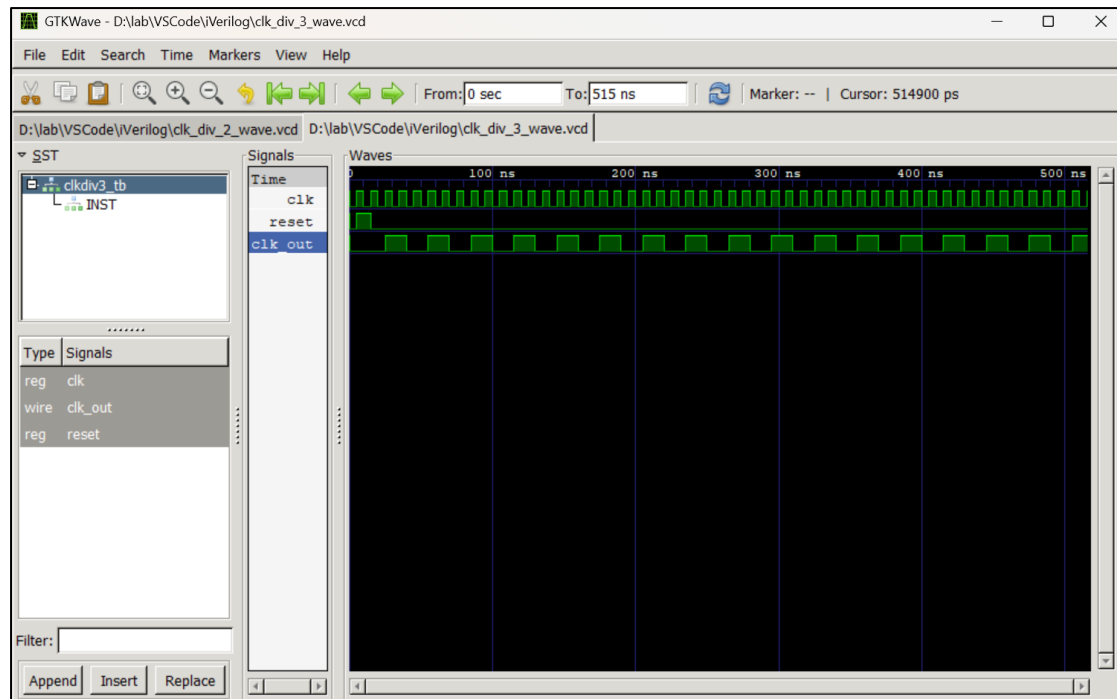
```

```

        $dumpvars(0, clkdiv3_tb);    // 輸出 clkdiv3_tb 模組及其底下的所有
層級訊號變數
    end
endmodule

```

Simulation Result :



波型解釋:

1. 重置期 (reset = 1 時): 當 reset 被拉高時, pos_count 與 neg_count 都會被清為 0, 因此 clk_out 也維持在低電位 0。這個階段的目的, 是先讓除頻器進入明確的初始狀態, 避免一開始出現未知值或不穩定波形。
2. 正常除頻期 (reset = 0 之後):
 - 觀察 clk 的上升沿 (posedge clk), pos_count 會依序從 0 → 1 → 2 → 0 進行循環計數。
 - 觀察 clk 的下降沿 (negedge clk), neg_count 也會依序從 0 → 1 → 2 → 0 進行循環計數。
 - 當 pos_count == 2 或 neg_count == 2 時, clk_out 會被拉成高電位。
 - 由於正沿計數器與負沿計數器相差半個 clock 週期, 因此兩者的高電位區段可以互相補足, 使 clk_out 形成接近 50% Duty Cycle 的輸出波形。
 - 從波形可觀察到, clk 經過 3 個完整週期後, clk_out 才完成 1 個完整週期, 因此成功達成除以 3 的除頻效果。

13. 除以 3 除頻器 (Divide by 3) 設計概念與用途:

■ 設計概念:

- ◆ 奇數除頻的挑戰在於無法單純靠單一波沿觸發來達到 50% 的工作週期 (如果只用正沿, 輸出會是 1/3 高、2/3 低, 工作週期為 33.3%)。為了解決這個問題, 本設計採用了「雙沿交錯疊加法」: 設計兩個獨立的 2-bit 計數器, 一個在正沿 (posedge) 觸發計數 0, 1, 2, 另一個在負沿 (negedge) 觸發計數 0, 1, 2。這兩個計數器的波形在時間軸上剛好錯開了半個輸入時脈週期。當我們將兩者的狀態 (如 count == 2 的區間) 進行 OR 邏輯組合後, 就能將原本不足的半個週期補起來, 進而完美調製出 50% 工作週期的奇數除頻波形。

■ 應用:

- ◆ 通訊協定 (如 I2C, SPI) 或記憶體控制器 (DDR Controller) 中, 晶片內部硬體常需要在非對稱的時脈域之間交換資料, 奇數除頻器能提供精準且維持

50% 工作週期的時脈，確保資料在高速傳輸時不因工作週期失真（Duty Cycle Distortion）而導致取樣錯誤。

- ◆ 在數位系統中，不同模組不一定都需要使用最高頻率的系統時脈。除以 3 除頻器可將主時脈降低為三分之一，提供給低速控制模組、週期性控制邏輯或資料同步模組使用，避免所有電路都運作在高頻下，進而降低功耗與切換雜訊。
- ◆ 在七段顯示器、多工掃描顯示、鍵盤矩陣掃描或簡易顯示控制電路中，常需要比系統時脈更低的掃描頻率。除以 3 除頻器可作為掃描時序的一部分，用於控制掃描週期與資料切換時間。